

TEÓRICO 2/10

TDD – TEST DRIVEN DEVELOPMENT

Se suele confundir con pruebas o testing, piensan que es una metodología de prueba.

Es un método para desarrollar software (construir código)

Test Driven Development → desarrollo conducido por pruebas

Es una actividad práctica que está relacionada con lineamientos y decisiones de como yo quiero trabajar.

Actividades del Desarrollo de Software

1. Requerimientos
2. Análisis
3. Diseño
4. Implementación
5. Prueba
6. Despliegue

¿Cuál es la actividad más cara? El costo más representativo es el costo del trabajo, es decir, el costo del esfuerzo, medida en hs. personas lineales. Es por ello que la actividad más cara es la **Prueba**.

La Prueba se lleva entre el 30-50% del costo total del proyecto, dependiendo del presupuesto y el objetivo del proyecto. La mayoría no le dedica ni cerca del 30%, lo cual está mal. Si le dedico el 30% de la implementación para las Pruebas no le estoy dedicando en realidad el 30%.

Antes se ponían a pensar cómo hacer testing una vez que el producto ya está desarrollado, hasta que se dieron cuenta que no es necesario tener el software programado para probar el código, hay cosas que se pueden hacer previamente como, por ejemplo, diseñar o plantear los casos de prueba.

Los Casos de Prueba son más específicos que los escenarios. Los escenarios son el input para que podamos diseñar los CDP (Caso de Prueba).

Un **Caso de Prueba** es la identificación de situaciones concretas más los datos de seteo necesarios para poder determinar si uno o más criterios de aceptación se cumplen para el software. Necesitan casos concretos para darle valor a la US, entonces yo determino toda la situación de contexto que son un conjunto de situaciones que deben ser validas. Es el artefacto más importante que obtenemos de la actividad de Prueba.

Muchas veces solo se estima el desarrollo, pero no se tiene en cuenta la implementación y las pruebas.

Entonces surgió la pregunta: ¿Por qué no adelantar la planificación y desarrollo de los casos de prueba así cuando debo ejecutarlos, gane tiempo?

Empezó en el **desarrollo tradicional** la idea de adelantar el trabajo para pensar cómo vamos a probar el software que vamos a construir. “Pensemos el testing desde el principio”, por eso el agilismo viene con esa idea desde el inicio. Adelantar todo lo que se pueda en el pensamiento de testing, así una vez que tengo el código solo me queda la ejecución de esas pruebas.

Los desarrolladores diseñan las pruebas unitarias después de escribir código

El aseguramiento y control de calidad es más reactivo



Los desarrolladores diseñan las pruebas unitarias antes de escribir el código

El aseguramiento y control de calidad es más proactivo

No hay forma de probar todo lo que hay que probar, por eso priorizaremos.

El desarrollador en este ámbito de trabajo funcionaba como un cirujano, hace y se va, todo lo demás es problema de los otros. Termina de desarrollar (muchas veces al terminar ni hacen las pruebas unitarias, por eso surge el DoD) y se desentiende del código, por ello aparece el rol de DevOps (Desarrollo de operaciones) que el código que vos construís, vos te encargas de que funcione en producción y de hacer el mantenimiento que necesite.

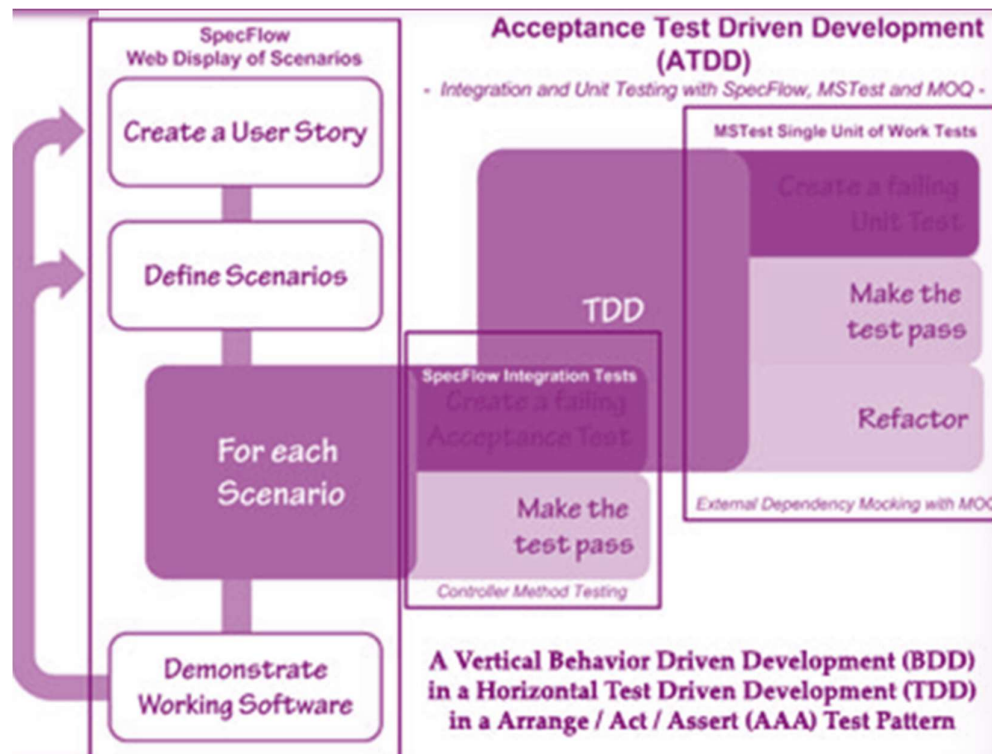
El agilismo lo que hace es impulsar la idea de que cuando trabajas con req., los mismos detallan respecto a cómo debe funcionar, para después en el análisis detallarlos (que, quien, como, etc.). El agilismo dice que vamos a trabajar de otra manera, según él, en la etapa de requerimientos sin detalle se puede trabajar, busca poner el detalle en donde realmente es imprescindible y no se puede evitar. “Requerimientos sin tanto detalle, y se detalla en las Pruebas”.

En el TDD primero pienso en las pruebas, y luego las corro automáticamente. Ya no se pierde el tiempo porque automatizamos.

El TDD evoluciona a **ATDD** (Aceptación), es decir, desarrollo conducido por pruebas de aceptación. Estas pruebas son:

- Prueba unitaria: se hace en el ambiente de implementación.
- Prueba de integración: si junto dos componentes que funcionan por separado, ¿anda bien igual? Genera un software que trabaja junto.
- Prueba de Sistema/Versión: tomo lo que se genera en la prueba de integración, ya es una versión. Están pensadas para ver verificación y validación de los RNF y RF.
- Prueba de UAT (Prueba de aceptación de usuario): es importante que las haga el usuario sí o sí. Están en condición de ponerlas en producción.

La mayoría de las empresas no pueden invertir en ambientes de prueba, y hacen las pruebas en los ambientes de producción.



El TDD es una técnica avanzada que involucra otras dos prácticas: Escribir las pruebas primero (Test First Development) y Refactorización (Refactoring). Para escribir las pruebas generalmente se utilizan las pruebas unitarias. Primero escribiremos casos de prueba, para luego correr las pruebas y ver si el componente pasa las pruebas pautadas.

#BENEFICIOS PPT

Tiene el ciclo Red-Green-Refactor. Basado en la idea de que el 1er test que correré debe fallar, para identificar que oportunidades de mejora tengo. Escribo un test case que falle, y esta falla permitirá identificar mejoras.

- Red: test fails
- Green: test passes
- Refactor: clean code + test

Las 3 leyes de TDD:

1. No escribir una línea de código hasta que no hayas escrito antes un test case que falla
2. No es necesario escribir más test de los necesarios para que falle. Con 1 alcanza
3. No vas a escribir más código en Producción que el necesario para que el test pase

Los desarrolladores tienen una tendencia de desarrollar cosas que nadie le pidió. Esta técnica nos ayuda a controlar esta situación, no escribas más de lo que hace falta para que pasen estas pruebas. Principio YAGNI. No hace falta que hagas muchos que fallen, con uno alcanza.

REFACTORING

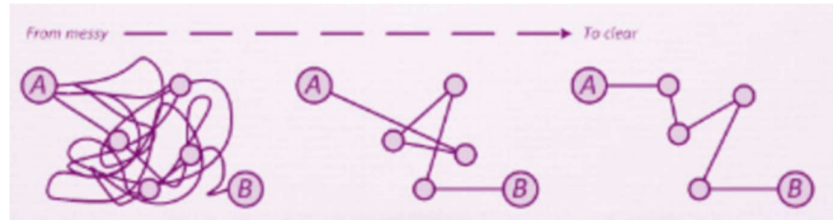
Antes hubo un factoring, sino no puedo llevarlo a cabo. Forma de desarrollar que apunta a mejorar la calidad interna del componente que compuse. Significa que no cambio las funcionalidades del componente, hará lo mismo que antes, solo que busco que lo haga mejor.

Es una forma disciplinada de introducir cambios reduciendo la posibilidad de introducir defectos.

¿Cómo hago mejor un componente?

Aplicando el nivel adecuado de acoplamiento y cohesión.

Lograré hacer refactoring



El refactoring lo suma el TDD para mejorar la calidad del software.

¿Cómo hacer Refactoring?

Nadie va a escribir un código mal propósito. El foco de TDD en la 1ra ronda es que las pruebas fallen, para salir rápidamente con un código para testear y hacer el ciclo. Una vez que la prueba fallo, escribo el código mínimo para que pase la prueba. Luego de esto, busco mejorar la calidad del código. Es un ciclo!!!



EJEMPLO PRÁCTICO:

Taxi-Mobile

Caso que debemos probar: Taxis libres (identificador verde) con chapa válida para un barrio existente

- Precondición (también conocido como seteo): todo lo que yo necesito tener disponible para poder ejecutar una determinada prueba. Establecen el estado inicial necesario para poder ejecutar correctamente los pasos de la prueba.

Buena práctica: definir la función de la forma:

def test_<<prueba que estamos haciendo>>():

por ejemplo: def test_mostrar_mapa_taxis_libres_en_barrio():

Dentro de la función, planteo las precondiciones

```
//PRECONDICIONES
usuario = {'nombre': 'Franco',
'apellido': 'Colapinto',
'permisos': 'Administrador de la central'}
usuario = iniciar_sesión
```

Luego defino los pasos (secuencia del CU). Tiene que haber una selección dentro de un menú de opciones. Estos pasos reflejan interacciones del usuario con el sistema y realizan el llamado a la US.

```
//PASOS DEL CASO DE PRUEBA
seleccionar_opcion_menu('Ver mapa')
```

Una vez que ejecuto esto, debo validar los resultados para definir si el test pasa o no pasa.
Estos resultados definen que debe devolver el sistema y valida que el comportamiento es correcto.

```
//RESULTADOS
mapa_mostrado = mostrar_mapa()
assert mapa_mostrado['ciudad'] == 'Córdoba', "El mapa mostrado no corresponde a la ciudad de Córdoba"
assert all(taxi['estado'] == 'Libre' for taxi in mapa_mostrado['taxis']), "No todos los taxis están en estado 'Libre'"
assert all(taxi['icono'] == 'verde' for taxi in mapa_mostrado['taxis']), "No todos los taxis tienen ícono verde"
```

Todos los casos de prueba deben tener por lo menos un assert

Assert es una palabra clave que se utiliza para verificar condiciones en las palabras. Si la condición del assert es falsa, la prueba falla

Assert all implica que se deben cumplir todas las condiciones para que cumpla la prueba

Para finalizar, debemos indicar si el test pasa o falla, para eso se utiliza el mensaje final. Si todas las verificaciones anteriores con assert son verdaderas, se imprimirá un mensaje que indica que la prueba es exitosa.

```
// Para cuando todos los casos son verdaderos
```